

Software Engineering Intro

Condensed Study Notes

Generated: 2026-05-25 00:12 UTC

Course Contents

This course will explore several key areas of software development, focusing on how software is designed, built, and the tools that support these processes.

Software Design delves into the fundamental principles and techniques for creating well-structured and maintainable software.

- **Software architecture:** This refers to the high-level structure of a software system, defining how its different parts fit together. Think of it like the blueprint for a building, showing the major rooms and how they connect.
- **Design patterns:** These are reusable solutions to common problems encountered in software design. They are like tried-and-true recipes that developers can adapt to build robust software.
- **OO analysis & Design:** This focuses on Object-Oriented (OO) principles for analyzing problems and designing software. It involves breaking down a system into 'objects' that represent real-world entities or concepts.
- **Design for re-use:** This principle emphasizes creating software components that can be easily used in multiple projects, saving time and effort.

Using APIs covers how to interact with existing software components and services.

- **API programming:** This involves writing code that communicates with Application Programming Interfaces (APIs). An API acts as a messenger that allows different software applications to talk to each other.
- **Class browsers and Related tools:** These are software tools that help developers understand and navigate the structure of code, particularly in object-oriented programming.
- **Component based computing:** This approach builds software by assembling independent, interchangeable units called components.

Software tools and Environment examines the various tools that assist in the software development lifecycle.

- **Requirements analysis and design modeling tools:** These tools help in understanding what a software system needs to do and in creating visual representations of its design.
- **Testing tools:** These are used to automatically check if software is working correctly and meets its specifications.
- **Tool integration mechanisms:** These are ways to connect different software tools so they can work together seamlessly.

The Term Software

Software is essentially a collection of instructions that tell a computer what to do. Think of it like a recipe:

- **Inputs:** The ingredients you start with.
- **Manipulation:** The steps in the recipe that transform the ingredients.
- **Outputs:** The final dish, which is the desired result in terms of what it does (its functions) and how well it does it (its performance).

These instructions are designed to achieve specific goals as determined by the person using the software (the user).

Beyond just the instructions, software also includes related documents, such as a software manual. These documents are crucial for helping users understand and effectively use the software system.

Today's software is comprised of

Software is more than just the code you see; it's a complete package that includes several key components.

At its core, software consists of:

- **Instructions (Computer Programs):** These are the fundamental commands that tell a computer what to do. When executed, they perform specific functions and achieve desired performance. Think of these like the individual steps in a recipe that guide you to bake a cake.
- **Data Structures:** These are organized ways of storing and managing information that allow programs to effectively manipulate data. Imagine them as different types of containers (like boxes, drawers, or shelves) designed to hold specific kinds of ingredients for your recipe, making them easy to find and use.
- **Documents:** Software also includes documentation that explains its operation and use. For example, documents might describe how to dispatch goods, and can detail optional features based on customer choices. This is like the recipe card itself, explaining how to follow the steps and what ingredients are needed.

Software is built with a specific **platform** in mind, which refers to the hardware and operating system it's designed to run on. Therefore, software is often described by its capabilities and the platform specifications required for it to function.

Beyond the core components, the software lifecycle involves:

- **Source Code:** This is the human-readable code written by programmers in languages like Java, VB, or Python. It's the blueprint for the software.
- **Executable:** These are compiled versions of the source code that can be run directly by the computer without needing the original source code. They can be shared and run independently.
- **Design Documents:** Before coding, various designs are created, such as input/output designs, UML diagrams, use case designs, class diagrams, and software architecture. These are like the architectural drawings for a building, planning out how everything will fit together.
- **Operations:** These are the expected functions and features of the software, guiding developers on how to build each part. They can be tailored to specific needs.
- **System Manuals:** These are crucial for users and include user guides, technical manuals, and complete system descriptions.

- **Installation:** Once developed, software needs to be installed on the user's system, a process that depends on the hardware and operating system.
- **Implementation:** After installation, the software is used in a real-world setting. This phase is critical to ensure it integrates smoothly with existing systems, using techniques like direct, pilot, phased, or parallel implementation.

The Term Engineering

The term 'engineering' broadly refers to the profession and activity of designing and creating things.

Adam et al. (2001) define engineering as the design of structures like roads and bridges, or the creation of machinery and equipment that results from applied knowledge.

Roger (2005) adds that engineering is more than just a field of study; it's an action word, a way of tackling problems. It can be understood as the act of making, achieving, or obtaining something through 'contrivance' or 'guile'.

- **Contrivance** refers to something invented or devised for a specific function, like a specialized machine.
- **Guile**, in this context, relates to finding solutions to problems, sometimes in a clever or cunning way, not necessarily involving deceit.

An engineering process involves setting clear **goals** that need to be achieved. These goals are often influenced by **constraints**, which are limitations or conditions that can affect the process. The engineer's role is to manage these factors through measurement, action, and process. This involves understanding needs and gathering necessary resources to reach the goals and produce a product or deliver a service.

In essence, engineering is about bringing about a desired outcome by carefully managing resources, processes, and potential limitations to achieve specific objectives.

Source Code vs. Executable in Visual Basic

This section provides a practical guide on how to distinguish between source code and an executable program when working with Visual Basic.

Think of it like a recipe:

- **Source Code:** This is like the written recipe itself. It contains the human-readable instructions and logic that a programmer writes. In Visual Basic, this is the code you see and edit in the development environment.
- **Executable:** This is like the finished cake, ready to be served and eaten. It's the program that the computer can directly understand and run. It's the result of processing the source code.

Visual Basic allows you to see and edit the source code, and then it can be compiled into an executable file that others can run without needing the Visual Basic development tools.

The Term Software Engineering

Software Engineering is essentially the application of engineering principles to the world of software development. It's not just about writing code; it's a systematic and disciplined approach to building software.

Think of it like building a bridge. You wouldn't just start piling up concrete and steel. You'd apply principles of physics, structural integrity, and planning to ensure the bridge is safe, functional, and cost-effective. Software engineering does the same for software.

Key aspects of Software Engineering include:

- **Cost-Effective Solutions:** Finding practical and affordable ways to solve problems using software.
- **Scientific Knowledge:** Applying established principles and knowledge to the creation of software.
- **Building Software Artifacts:** Creating software products (like applications, systems, etc.) that serve a purpose.
- **Analyzing User Needs:** Understanding what users require from a software product.
- **Designing, Constructing, and Testing:** The core activities of creating and verifying software.
- **Systematic, Disciplined, and Quantifiable Approach:** Using structured methods that can be measured and followed consistently.

This approach is particularly important for larger and more complex software systems, focusing on best practices and robust processes to ensure quality and reliability throughout the software's lifecycle, including its development, operation, and maintenance.

Computer Science VS Software Engineering

Computer Science is primarily the study of how computers function, approached from a theoretical and mathematical viewpoint. It delves into topics like algorithms, data structures, and operating systems.

Software Engineering, on the other hand, focuses on the practical aspects of building software systems. This includes managing projects, designing software architecture, handling risks, ensuring quality through testing, and understanding the entire lifecycle of software development and maintenance.

While both fields teach programming fundamentals, they offer different paths:

- **Choose Computer Science if:** You enjoy mathematics and logic, or if you're interested in specialized areas like artificial intelligence (AI), machine learning, security, or graphics. It's like studying the fundamental laws of physics.
- **Choose Software Engineering if:** You prefer a hands-on approach and want to learn the complete process of how software is created and kept running. This is akin to being an architect and builder of complex structures, ensuring they are functional, reliable, and maintainable over time.

OR

Software Engineering is about using scientific knowledge and structured methods to create software solutions that are practical and cost-effective.

Its main goals are to produce software that:

- Is free from errors (fault-free)
- Meets the needs of the users
- Is delivered on time
- Stays within budget

Think of it like building a bridge. You wouldn't just start piling up materials; you'd use engineering principles, scientific knowledge about materials and stresses, and a systematic plan to ensure the bridge is safe, functional, and built efficiently. Software Engineering applies this same disciplined approach to creating software.

The term "Software Engineering" itself represents a movement and a collection of techniques designed to make the process of developing software more organized and predictable.

The key differences are

The primary distinctions between Computer Science and Software Engineering lie in their curriculum structure and focus.

Computer Science:

- Offers more flexibility with electives in advanced topics.
- You can explore areas like security, computer vision, machine learning, and database management.

Software Engineering:

- Features more required courses in core software engineering principles.
- These include essential areas such as software testing, design, and specifying software requirements.

Importance of Software

Software has become a powerful force, acting as the engine behind many crucial aspects of our modern world.

Here's why it's so important:

- **Business Decision Making:** Software is the primary tool that drives how businesses make decisions. Think of it like a sophisticated dashboard for a car, providing all the vital information needed to steer effectively.
- **Scientific and Engineering Advancement:** It forms the foundation for modern scientific research and helps engineers solve complex problems. It's like the advanced calculator and simulation tool that allows scientists and engineers to explore new ideas and build new things.
- **Embedded Systems:** Software is integrated into a vast array of systems we encounter daily, including transportation (like your car's navigation), medical equipment, telecommunications networks, military technology, industrial processes, entertainment devices, and office products. It's the invisible brain making these systems work.
- **Pervasive Impact:** Software affects nearly every facet of our lives, deeply influencing our commerce, culture, and daily activities. Its societal and cultural impact is significant and continues to grow.

As software's importance increases, the software community is constantly working on developing new technologies to make building high-quality programs easier, faster, and more cost-effective.